エンジニア研修 ・ 2026年度版

テストコード入門

動くことを、コードで確かめ続ける

JUnit　AssertJ　Mockito　Testcontainers

© 2026 by yo1000 | Yoichi KIKUCHI is licensed under CC BY-ND 4.0

この講義の流れ

テストコードを3部で学ぶ

	部	内容
01	テストコードの概要と構成要素	テストを書く理由、テストの種類、テストを支える3つの道具
02	JUnit による基本的なテスト	JUnit の基本、パラメタライズドテスト、アサーション、モック
03	Spring プロジェクトのテスト	JDBC・JPA・RestController・KafkaListener（Testcontainers）

第2部と第3部には演習があります。サンプルプロジェクトのコードを読み、テストを実行し、演習用のテストクラスに書き足しながら進めます。

はじめに

演習準備：研修のリポジトリを動かす

前提	内容
Java 21	Java 21（JDK）が、環境にインストールされていることを前提とします
ターミナル	WSL・Linux・macOS など、ターミナルからコマンドを実行できる環境を前提とします
Docker	第3部（spring-test）のテストで使います。Testcontainers が PostgreSQL と Kafka のコンテナを起動します

まず、Java 21 が使えることを確かめます。表示されるバージョンが 21 であれば準備はできています。

```
java -version
```

テストは、リポジトリのフォルダで Maven Wrapper（./mvnw）を使って実行します。次のコマンドは、3つのモジュールすべてのテストを実行します。

```
./mvnw test
```

Maven Wrapper（mvnw）とは

プロジェクトで決めたバージョンの Maven を、初回の実行時にダウンロードして使うスクリプトです。Maven を自分でインストールしなくてよく、全員が同じバージョンの Maven でテストを実行できます。

はじめに

テストクラスを指定して実行する

-pl にモジュールのフォルダ名を、-Dtest にテストクラスの名前を指定すると、そのクラスのテストだけを実行できます。

```
./mvnw test -pl unit-test -Dtest=PriceCalculatorTest
```

実行結果（抜粋）

```
[INFO] -------------------------------------------------------
[INFO]  T E S T S
[INFO] -------------------------------------------------------
[INFO] Running PriceCalculator：支払金額の計算
（中略）
[INFO] Tests run: 5, Failures: 0, Errors: 0, Skipped: 0
[INFO] BUILD SUCCESS
```

項目	意味
Tests run	実行したテストの数
Failures	期待した結果にならなかった（アサーションに失敗した）テストの数
Errors	想定外の例外が起きたテストの数
Skipped	実行しなかった（@Disabled などの）テストの数

はじめに

リポジトリの構成：部ごとのモジュール

モジュール	内容	演習
overview	第1部：スライドのコード（テストの形・3つの道具・アサーションの比較）	—
unit-test	第2部：Spring を使わない Java のクラス（支払金額・送料・注文）と、そのテスト	演習1〜4
spring-test	第3部：Spring Boot のアプリケーション（商品・会員）と、そのテスト	演習5〜7

ファイル	役割
pom.xml	3つのモジュールをまとめて実行するための pom（modules だけを書く）
overview・unit-test・spring-test の pom.xml	各モジュールの pom。ルートの pom を parent にしないので、モジュールだけでもビルドできる
spring-test/src/main/resources/schema.sql	テーブル定義（products・members）
mvnw・mvnw.cmd・.mvn/	Maven Wrapper。./mvnw で Maven を実行する

テストコードは src/test/java に、テスト対象と同じパッケージで置きます。クラス名は「テスト対象のクラス名 + Test」が慣例です。スライドの下には、そのコードがあるファイルを示しています。

01

テストコードの概要と構成要素

テストを書く理由と、テストを支える道具

テストコードの概要

テストコード：コードの動きを確かめるコード

作ったメソッドが正しく動くかを、人が画面や出力を見て確かめる代わりに、コードで確かめます。

人が確かめる

```
// main で呼び出して、出力を目で見る
int total = calculator.calculate(1000, 2, MemberRank.GOLD);
System.out.println(total);
```

出力

1980

テストコードで確かめる

```
// 期待する値をコードに書いておく
int total = calculator.calculate(1000, 2, MemberRank.GOLD);
Assertions.assertEquals(1980, total);
```

結果

成功（または失敗とその理由）

ポイント

- 期待する結果をコードに書くので、正しいかどうかの判断を人がしなくてよい
- 何度でも、何百件でも、同じ手順で確かめられる

overview/src/main/java/com/example/testing/overview/ManualCheck.java
overview/src/test/java/com/example/testing/overview/PriceCalculatorTest.java

テストコードの概要

テストコードを書く理由

正しく動くことを確かめる

作ったときに、仕様どおりに動くかを確かめる。境界値や異常な入力など、手で試しにくいケースも確かめられる。

壊れていないことを確かめ続ける

機能追加やリファクタリングのあと、すべてのテストを実行し直せば、既存の動きが変わっていないことがすぐにわかる（回帰テスト）。

使い方と仕様を書き残す

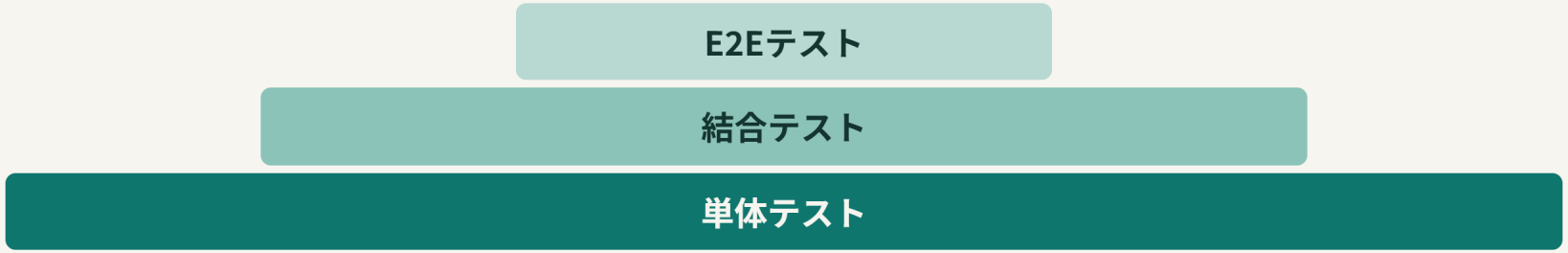
テストには「この入力ならこの結果」が書かれている。どう呼び出し、何が返るのかを示す、実行できるドキュメントになる。

一番の価値は2つ目です。テストがあれば、コードを安心して書き換えられます。

テストコードの概要

テストの種類：確かめる範囲で分ける

種類	確かめる範囲	速さ	数
単体テスト	1つのクラスやメソッド。依存先は偽物に置き換えることが多い	速い	多い
結合テスト	複数の部品のつながり。データベースやメッセージングなども含む	やや遅い	中くらい
E2Eテスト	画面や API の入口から出口まで、システム全体	遅い	少ない



テストピラミッド：速くて安定している単体テストを土台に多く書き、遅い E2E テストは要所に絞る、という考え方です。

テストコードの概要

テストの形：準備・実行・検証

テストメソッドは、多くの場合、次の3つの段落で書けます。Arrange-Act-Assert（AAA）と呼ばれる書き方です。

```
@Test
void regularMember() {
    // Arrange（準備）：テスト対象と入力値を用意する
    PriceCalculator calculator = new PriceCalculator();
    int unitPrice = 1000;
    int quantity = 1;

    // Act（実行）：テスト対象のメソッドを1回呼ぶ
    int actual = calculator.calculate(unitPrice, quantity, MemberRank.REGULAR);

    // Assert（検証）：結果が期待どおりかを確かめる
    Assertions.assertEquals(1100, actual);
}
```

ポイント

・1つのテストメソッドでは、1つの振る舞いを確かめる

・テスト対象も Arrange で組み立てる。何をどう用意したかが、そのテストを読むだけでわかる

・Given-When-Then（前提・操作・結果）とも呼ぶ。BDD（振る舞い駆動開発）から広まった呼び方で、Java のコードでは // given // when // then とコメントを書くことも多い。考え方は同じ

テストコードの構成要素

テストコードを支える3つの道具

テスティングフレームワーク

テストメソッドを見つけて実行し、成功・失敗を集計して報告する。テストの前後処理や、テストの組み立て方も提供する。

アサーションライブラリ

「検査される値が期待値どおりか」を判定し、違っていればわかりやすいメッセージでテストを失敗させる。

モッキングライブラリ

テスト対象が使う部品（依存先）を、振る舞いを自由に決められる偽物（モック）に置き換える。呼び出されたかどうかも確かめられる。

テストコードの構成要素

1つのテストの中で、3つの道具が役割を分けている

```
@Test // JUnit：テストメソッドの目印
void placeOrder() {
    // Mockito：モックを作り、振る舞いを決める
    StockRepository stockRepository = Mockito.mock(StockRepository.class);
    Mockito.when(stockRepository.findStock("A001")).thenReturn(10);
    OrderService orderService =
            new OrderService(stockRepository, new PriceCalculator(), Mockito.mock(OrderNotifier.class));

    Order order = orderService.placeOrder("A001", 1000, 2, MemberRank.GOLD);

    Assertions.assertThat(order.totalPrice()).isEqualTo(1980); // AssertJ：判定する
    Mockito.verify(stockRepository).decrease("A001", 2); // Mockito：呼び出しを確かめる
}
```

道具	このコードでの役割	ライブラリ
テスティングフレームワーク	@Test	JUnit
アサーションライブラリ	Assertions.assertThat(…).isEqualTo(…)	AssertJ
モッキングライブラリ	Mockito.mock(…)・Mockito.when(…)・Mockito.verify(…)	Mockito

overview/src/test/java/com/example/testing/overview/OrderServiceTest.java

テスティングフレームワーク：Java では JUnit

名前	特徴
JUnit	Java のテスティングフレームワークの標準。現在の JUnit 5・JUnit 6 は「JUnit Jupiter」という API でテストを書く
TestNG	テストの依存関係やグループ分けなど、細かな制御が得意。JUnit 4 の時代に広まった
Spock	Groovy でテストを書く。given:・when:・then: のブロックで読みやすく書ける

JUnit のバージョン

- JUnit 4 と JUnit 5 以降では、パッケージもアノテーションも違う（org.junit と org.junit.jupiter）
- JUnit 6（2025年）は JUnit 5 と同じ Jupiter の API で書ける。この講義のコードもそのまま動く
- インターネット上の古い記事は JUnit 4 で書かれていることがある。import を見て見分ける

アサーションライブラリ：同じ判定を、3つの書き方で

ライブラリ	書き方
JUnit 標準	Assertions.assertEquals(1980, total);
AssertJ	Assertions.assertThat(total).isEqualTo(1980);
Hamcrest	MatcherAssert.assertThat(total, Matchers.is(1980));

ポイント

- JUnit 標準：追加のライブラリが要らない。期待値から書くので、期待値と検査される値の順番を取り違えやすい
- AssertJ：Assertions.assertThat(検査される値) のあとに「.」で判定をつなげる。エディタの補完で判定を選べる
- Hamcrest：判定を「マッチャー」の組み合わせで書く。AssertJ より前に広く使われていた

現在の Java では AssertJ が広く使われています。この講義でも、第2部の途中から AssertJ を使います。

モッキングライブラリ：Java では Mockito

名前	特徴
Mockito	Java のモッキングライブラリの標準。Mockito.when(…).thenReturn(…) と Mockito.verify(…) で書く
EasyMock	「期待する呼び出しを先に記録し、あとで再生する」書き方。Mockito より前から使われている
MockK	Kotlin 向け。Kotlin の言語機能（拡張関数など）にも対応する

モックにするのは、テスト対象のクラスが依存しているインスタンスです。

テスト対象	モックにする依存先	この講義での例
Controller	ApplicationService	ProductController → ProductService（第3部）
ApplicationService	Repository	OrderService → StockRepository（第2部）

ポイント

- 依存先をモックにすると、依存先の動きに左右されず、テスト対象の振る舞いだけを確かめられる
- 計算だけのクラスや値オブジェクトは、モックにせず本物を使う（OrderService が使う PriceCalculator など）

テストコードの構成要素

JUnit + AssertJ + Mockito が、ほぼ定番の組み合わせ

Spring Boot のプロジェクトを作ると、spring-boot-starter-test が依存関係に入ります。この中に、3つの道具がそろっています。

```
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-test</artifactId>
    <scope>test</scope>
</dependency>
```

含まれるもの	役割
JUnit Jupiter	テスティングフレームワーク
AssertJ / Hamcrest	アサーションライブラリ
Mockito	モッキングライブラリ
Spring Test / Spring Boot Test	Spring のアプリケーションをテストするための仕組み
JSONassert / JsonPath	JSON の判定

spring-test/pom.xml

テストコードの構成要素

Spring Boot の標準が、現場の標準になっていく

Java のサーバーサイド開発では Spring Boot が広く使われています。そのため、Spring Boot が標準で採用したライブラリが、そのまま多くのプロジェクトで使われるようになります。

時期	Spring Boot の変更	その後
2019年	Spring Boot 2.2 で、標準を JUnit 4 から JUnit 5 に変更	JUnit 5 への移行が一気に進んだ
2020年	Spring Boot 2.4 で、JUnit 4 を動かす仕組みを標準から外す	JUnit 4 を使い続けるには、自分で追加する必要が出た
2023年	Spring Boot 3.1 で、Testcontainers との連携（@ServiceConnection）を追加	コンテナを使ったテストが書きやすくなった
2025年	Spring Boot 4.0 で、JUnit 6 を採用	最新の JUnit がすぐに使われ始める

ライブラリを選ぶときは「Spring Boot が標準で何を使っているか」が、大きな判断材料になります。

最近のテスト

コンテナを使って、本物に近い環境でテストする

データベースや Kafka を使うコードのテストでは、以前はメモリ上で動く代わりのもの（H2 など）がよく使われていました。最近は、本番と同じソフトウェアをコンテナで起動してテストすることが増えています。

メモリ上の代わりのもので確かめる

特徴

・H2 や EmbeddedKafka を JVM の中で動かす

・速く、Docker も要らない

・本番と違うソフトウェアなので、SQL の方言や設定の違いでテストと本番の結果が食い違うことがある

コンテナで本物を起動して確かめる

特徴

・PostgreSQL や Kafka の Docker イメージを起動する

・Docker が必要で、起動に数秒かかる

・本番と同じソフトウェアなので、テストが通れば本番でも同じように動くことが期待できる

テストのたびにコンテナを起動・停止してくれるライブラリが Testcontainers です。

Testcontainers：テストの中でコンテナを動かす

	Testcontainers がすること
1	テストクラスの実行前に、指定した Docker イメージのコンテナを起動する
2	起動したコンテナの接続先（ホスト・ポート・ユーザー名など）をテストに渡す
3	テストが、そのコンテナに接続して実行される
4	テストクラスの実行が終わったら、コンテナを停止して削除する

```
@Container
@ServiceConnection
static PostgreSQLContainer postgres = new PostgreSQLContainer("postgres:17-alpine");
```

ポイント

- ポート番号は空いているものが自動で選ばれるので、他のテストやアプリとぶつからない
- @ServiceConnection を付けると、Spring Boot が接続先を自動で設定する（第3部）

第1部のまとめ

テストコードの概要と構成要素

テストコードは、動きを確かめるコード

期待する結果をコードに書いておけば、何度でも同じ基準で確かめられる。変更しても壊れていないことを確かめ続けられる。

3つの道具を組み合わせる

テスティングフレームワーク（JUnit）・アサーションライブラリ（AssertJ）・モッキングライブラリ（Mockito）。Spring Boot の標準もこの組み合わせ。

本物に近い環境でも確かめる

データベースやメッセージングは、Testcontainers でコンテナを起動し、本番と同じソフトウェアで確かめる。

02

JUnit による基本的なテスト

JUnit・AssertJ・Mockito を手を動かして覚える

テスト対象：支払金額を計算する PriceCalculator

```
public class PriceCalculator {

    private static final int TAX_RATE_PERCENT = 10;

    public int calculate(int unitPrice, int quantity, MemberRank rank) {
        if (unitPrice < 0) {
            throw new IllegalArgumentException("単価は0以上にしてください: " + unitPrice);
        }
        if (quantity < 1) {
            throw new IllegalArgumentException("数量は1以上にしてください: " + quantity);
        }
        int subtotal = unitPrice * quantity;
        int discounted = subtotal * (100 - rank.discountPercent()) / 100;
        return discounted * (100 + TAX_RATE_PERCENT) / 100;
    }
}
```

会員ランク（MemberRank）	REGULAR	SILVER	GOLD
割引率	0%	5%	10%

unit-test/src/main/java/com/example/testing/basic/PriceCalculator.java

JUnit の基本

最初のテストを読む

```
import org.junit.jupiter.api.Assertions;

class PriceCalculatorTest {
    @Test
    void regularMember() {
        // Arrange
        PriceCalculator calculator = new PriceCalculator();

        // Act
        int actual = calculator.calculate(1000, 1, MemberRank.REGULAR);

        // Assert
        Assertions.assertEquals(1100, actual);
    }
}
```

ポイント

- @Test を付けたメソッドが、1つのテストになる（public でなくてよい）
- テスト対象（calculator）は、テストメソッドの中の Arrange で組み立てる
- Assertions.assertEquals(期待値, 検査される値) の順に書く。Assertions は org.junit.jupiter.api パッケージのクラス

unit-test/src/test/java/com/example/testing/basic/PriceCalculatorTest.java

JUnit の基本

よく使うアノテーション

アノテーション	意味
@Test	テストメソッドの目印
@DisplayName("…")	テスト結果に表示する名前。日本語で書ける
@ParameterizedTest	値の組を変えながら、同じテストを繰り返す。値は @CsvSource で渡す（このあと詳しく扱う）
@BeforeEach / @AfterEach	各テストメソッドの前／後に毎回実行する
@BeforeAll / @AfterAll	クラスの最初／最後に1回だけ実行する（static メソッドに付ける）
@Disabled("理由")	そのテストを実行しない。理由を書いておく
@Nested	内部クラスでテストをグループに分ける

@ParameterizedTest・@CsvSource は org.junit.jupiter.params、そのほかは org.junit.jupiter.api パッケージにあります。JUnit 4 の org.junit.Test などと混ぜないようにします。

JUnit の基本

テストメソッドごとに、新しいインスタンスが作られる

JUnit は、テストメソッドを1つ実行するたびに、テストクラスのインスタンスを作り直します。

実行の流れ

```
new PriceCalculatorTest()   →  regularMember()
new PriceCalculatorTest()   →  goldMember()
new PriceCalculatorTest()   →  roundDown()
```

ポイント

・あるテストの結果が、ほかのテストに影響しないようにするための仕組み

・テストの実行順は決まっていない。テスト同士が順番に頼らないように書く

・テスト対象は、各テストの Arrange で組み立てる。テストごとに渡すモックや状態が変わるので、@BeforeEach で一律に作ってよい場面は限られる

JUnit の基本

@DisplayName と @Nested で、結果を読みやすくする

```
@DisplayName("PriceCalculator：支払金額の計算")
class PriceCalculatorTest {
    @Test
    @DisplayName("ゴールド会員は10%引きのあとに消費税が加わる")
    void goldMember() {
        PriceCalculator calculator = new PriceCalculator();
        int actual = calculator.calculate(1000, 2, MemberRank.GOLD);
        Assertions.assertEquals(1980, actual);
    }
    @Nested
    @DisplayName("入力値が不正なとき")
    class InvalidInput {
        @Test
        @DisplayName("数量が0なら IllegalArgumentException")
        void quantityZero() { ... }
    }
}
```

IDE のテスト結果の表示（イメージ）

PriceCalculator：支払金額の計算
├ ゴールド会員は10%引きのあとに消費税が加わる ✓
└ 入力値が不正なとき
　└ 数量が0なら IllegalArgumentException ✓

unit-test/src/test/java/com/example/testing/basic/PriceCalculatorTest.java

JUnit の基本

例外が投げられることを確かめる：assertThrows

```
@Test
@DisplayName("数量が0なら IllegalArgumentException")
void quantityZero() {
    PriceCalculator calculator = new PriceCalculator();

    IllegalArgumentException e = Assertions.assertThrows(IllegalArgumentException.class,
            () -> calculator.calculate(1000, 0, MemberRank.REGULAR));

    Assertions.assertEquals("数量は1以上にしてください: 0", e.getMessage());
}
```

ポイント

- 第1引数に期待する例外の型、第2引数に例外が起きるはずの処理をラムダ式で渡す
- 例外が起きなかったり、別の型の例外だったりすると、テストは失敗する
- 戻り値で受け取った例外から、メッセージなども確かめられる

よくある間違い

calculator.calculate(…) をラムダ式で包まずに直接書くと、assertThrows に渡す前に例外が起きてしまい、テストはエラーになります。

unit-test/src/test/java/com/example/testing/basic/PriceCalculatorTest.java

JUnit の基本

テストが失敗すると、何が違うのかが表示される

期待値を、わざと 1000 に書き換えて実行してみます。

```
Assertions.assertEquals(1000, actual); // 正しくは 1100
```

```
./mvnw test -pl unit-test -Dtest=PriceCalculatorTest
```

実行結果（抜粋）

```
[ERROR] Failures:
[ERROR]   PriceCalculatorTest.regularMember:26 expected: <1000> but was: <1100>
[INFO]
[ERROR] Tests run: 5, Failures: 1, Errors: 0, Skipped: 0
[INFO] BUILD FAILURE
```

ポイント

- expected が期待値、but was が検査された値。どのテストの何行目かもわかる
- 新しく書いたテストは、一度わざと失敗させて「本当に確かめているか」を見るとよい

unit-test/src/test/java/com/example/testing/basic/PriceCalculatorTest.java

JUnit の基本

何を確かめるか：境界の両側を選ぶ

演習で使う ShippingFeeCalculator（送料の計算）の仕様です。

条件	送料
購入金額が 5,000 円以上	0 円
購入金額が 5,000 円未満	500 円
離島への配送	上記に 1,000 円を加える
購入金額が 0 円未満	IllegalArgumentException

入力を、結果が同じになるグループに分け（同値分割）、グループの境目（境界値）の両側を確かめます。

購入金額	…	4998	4999	5000	5001	…
送料		500 円	500 円	0 円	0 円	

結果が変わる境目の両側、4,999 円と 5,000 円を必ず確かめます。

演習1 ・ JUnit の基本

送料の計算をテストする

ShippingFeeCalculatorTest に、次の3つのテストメソッドを追加してください。それぞれに @DisplayName も付けます。

- 購入金額が 5,000 円ちょうどなら、送料が 0 円になる
- 離島への配送で、購入金額が 4,999 円なら、送料が 1,500 円になる
- 離島への配送で、購入金額が 5,000 円なら、送料が 1,000 円になる

ファイル：unit-test/src/test/java/com/example/testing/basic/ShippingFeeCalculatorTest.java（「購入金額が5,000円未満なら送料は500円」のテストは書いてあります。calculate(購入金額, 離島かどうか) で呼び出します。テスト対象は、書いてあるテストと同じく各テストの中で new します）

```
./mvnw test -pl unit-test -Dtest=ShippingFeeCalculatorTest
```

期待する結果

```
[INFO] Tests run: 4, Failures: 0, Errors: 0, Skipped: 0
```

演習2 ・ JUnit の基本

不正な購入金額で例外になることをテストする

ShippingFeeCalculatorTest に、購入金額が -1 円なら IllegalArgumentException が投げられることを確かめるテストを追加してください。

あわせて、例外のメッセージが「購入金額は0以上にしてください: -1」であることも確かめます。

ヒント：Assertions.assertThrows(例外の型.class, () -> 例外が起きる処理) の戻り値から getMessage() でメッセージを取り出す

```
./mvnw test -pl unit-test -Dtest=ShippingFeeCalculatorTest
```

期待する結果

```
[INFO] Tests run: 5, Failures: 0, Errors: 0, Skipped: 0
```

パラメタライズドテスト

同じ形のテストが並んだら

```
@Test
void regular() {
    PriceCalculator calculator = new PriceCalculator();

    int actual = calculator.calculate(1000, 1, MemberRank.REGULAR);

    Assertions.assertEquals(1100, actual);
}

// silver()・gold() … と、値だけが違う同じ形のテストが続く
```

違うのは、入力と期待値だけです。パラメタライズドテストを使うと、テストメソッドを1つにして、値の組を表のように並べて書けます。

@Test の代わりに @ParameterizedTest を付け、値の組は @CsvSource で渡します。値の組ごとに、1つのテストとして実行されます。

unit-test/src/test/java/com/example/testing/basic/PriceCalculatorRepeatedTest.java

パラメタライズドテスト

@CsvSource：値の組を表のように並べる

```
@ParameterizedTest(name = "単価{0}円 × {1}個、{2} → {3}円")
@CsvSource(delimiter = '|', textBlock = """
        # unitPrice | quantity | rank    | expected
               1000 |        1 | REGULAR |     1100
               1000 |        3 | SILVER  |     3135
               1000 |        2 | GOLD    |     1980
                999 |        1 | SILVER  |     1043
                  0 |        5 | GOLD    |        0
        """)
void calculate(int unitPrice, int quantity, MemberRank rank, int expected) {
    PriceCalculator calculator = new PriceCalculator();

    int actual = calculator.calculate(unitPrice, quantity, rank);

    Assertions.assertEquals(expected, actual);
}
```

テスト結果の表示（イメージ）

```
├─ 単価1000円 × 1個、REGULAR → 1100円  ✓
├─ 単価1000円 × 3個、SILVER → 3135円  ✓
（以下略）
```

unit-test/src/test/java/com/example/testing/basic/PriceCalculatorParameterizedTest.java

パラメタライズドテスト

@CsvSource の書き方

書き方	意味
delimiter = '|'	列の区切りを「|」にする。値の中のカンマと混ざらず、表のように読める
textBlock = """ … """	テキストブロックに、1行に1組ずつ書く
# で始まる行	コメント。列の名前（メソッドの引数名）を書いておくと読みやすい
値の前後の空白	無視される。空白で桁をそろえられる
値の型	メソッドの引数の型（int・boolean・enum など）に自動で変換される
'…'	空白やカンマを含む文字列はシングルクォートで囲む。'' は空文字
name = "…{0}…"	テスト結果の表示名。{0}・{1} … は何番目の値か

ほとんどのパラメタライズドテストは、この @CsvSource の書き方だけで書けます。

パラメタライズドテスト

@CsvSource で null・空文字を渡す

```
@ParameterizedTest(name = "[{0}] → {1}")
@CsvSource(delimiter = '|', nullValues = "null", textBlock = """
        # code  | expected
          A001  | true
          null  | false
                | false
          ''    | false
          '   ' | false
        """)
void isValid(String code, boolean expected) {
    ProductCodeValidator validator = new ProductCodeValidator();

    boolean actual = validator.isValid(code);

    Assertions.assertEquals(expected, actual);
}
```

書き方	引数に渡される値
null	null（nullValues = "null" を指定したとき）
（何も書かない）	null（nullValues を指定しなくても）
''	空文字（""）
' '	空白だけの文字列（" "）。クォートの中の空白は取り除かれない

unit-test/src/test/java/com/example/testing/basic/ProductCodeValidatorTest.java

パラメタライズドテスト

そのほかの値のソース

@CsvSource のほかにも、値のソースがあります。ここでは名前だけ紹介します。

アノテーション	渡せる値
@ValueSource	1つの値の並び（引数が1つのとき）
@EnumSource	enum の定数すべて
@NullSource / @EmptySource	null／空文字・空のリスト
@CsvFileSource	CSV ファイルの行
@MethodSource	メソッドが返す値の組。オブジェクトやリストなど、CSV の文字列では表せない値も渡せる

@MethodSource を使うのは、@CsvSource で表せない複雑な値が引数に必要になったときです。使う場面はまれなので、必要になったときに調べれば十分です。

演習3 ・ パラメタライズドテスト

送料のテストを @CsvSource で書き直す

ShippingFeeCalculatorParameterizedTest に、@ParameterizedTest と @CsvSource を使ったテストメソッドを1つ書き、次の5組を確かめてください。

ヒント：@CsvSource(delimiter = '|', textBlock = """ … """) に「購入金額 | 離島かどうか | 期待値」を1行ずつ書き、メソッドの引数を (int, boolean, int) にする

購入金額	離島かどうか	期待する送料
0	false	500
4999	false	500
5000	false	0
4999	true	1500
5000	true	1000

```
./mvnw test -pl unit-test -Dtest=ShippingFeeCalculatorParameterizedTest
```

期待する結果

```
[INFO] Tests run: 5, Failures: 0, Errors: 0, Skipped: 0
```

アサーション ・ AssertJ

AssertJ：検査される値から書き始める

JUnit 標準：期待値から書く

```
import org.junit.jupiter.api.Assertions;

Assertions.assertEquals(1980, total);
```

特徴

・「1980 と total が等しい」と読む

・第1引数が期待値、第2引数が検査される値。役割は引数の順番だけで決まる

・順番を取り違えても動くが、失敗メッセージの expected と but was が逆になる

AssertJ：検査される値から書く

```
import org.assertj.core.api.Assertions;

Assertions.assertThat(total).isEqualTo(1980);
```

特徴

・「total が 1980 と等しい」と読む

・Assertions.assertThat(検査される値) で始め、「.」のあとに判定と期待値を書く

・「.」を打つと、その型で使える判定がエディタの補完に出てくる

JUnit と AssertJ には、どちらも Assertions という名前のクラスがあります。import するパッケージ（org.junit.jupiter.api と org.assertj.core.api）で見分けます。

overview/src/test/java/com/example/testing/overview/JUnitAssertionTest.java
overview/src/test/java/com/example/testing/overview/AssertJAssertionTest.java

アサーション ・ AssertJ

数値と文字列の判定

```
int total = new PriceCalculator().calculate(1000, 2, MemberRank.GOLD);

Assertions.assertThat(total)
        .isEqualTo(1980)
        .isPositive()
        .isBetween(1000, 2000);

Order order = new Order("A001", 2, 1980); // 注文（商品コード・数量・支払金額）のレコード
String text = order.toString();

Assertions.assertThat(text)
        .startsWith("Order[")
        .contains("productCode=A001")
        .endsWith("totalPrice=1980]");
```

ポイント

- 判定は「.」でつなげて、いくつも続けて書ける
- 途中の判定で失敗すると、そこでテストが止まり、失敗した判定が報告される
- Order は、注文（商品コード・数量・支払金額）を表すレコード。レコードの toString は「Order[項目=値, …]」の形になる

アサーション ・ AssertJ

リストの判定と、要素の取り出し（extracting）

```
List<Order> orders = List.of(
        new Order("A001", 2, 1980),
        new Order("B002", 1, 1100));

Assertions.assertThat(orders)
        .hasSize(2)
        .extracting(Order::productCode)
        .containsExactly("A001", "B002");

Assertions.assertThat(orders)
        .extracting(Order::productCode, Order::totalPrice)
        .containsExactly(
                Assertions.tuple("A001", 1980),
                Assertions.tuple("B002", 1100));
```

メソッド	判定の内容
contains(…)	指定した要素を含む（順番・ほかの要素は問わない）
containsExactly(…)	指定した要素だけを、その順番で含む
containsExactlyInAnyOrder(…)	指定した要素だけを含む（順番は問わない）

アサーション ・ AssertJ

例外と Optional の判定

```
PriceCalculator calculator = new PriceCalculator();

Assertions.assertThatThrownBy(() -> calculator.calculate(1000, 0, MemberRank.REGULAR))
        .isInstanceOf(IllegalArgumentException.class)
        .hasMessage("数量は1以上にしてください: 0");
```

```
Assertions.assertThat(Optional.of("GOLD")).hasValue("GOLD");
Assertions.assertThat(Optional.empty()).isEmpty();

Optional<Order> found = Optional.of(new Order("A001", 2, 1980));
Assertions.assertThat(found)
        .hasValueSatisfying(o -> Assertions.assertThat(o.totalPrice()).isEqualTo(1980));
```

ポイント

- assertThatThrownBy：例外の型とメッセージを、続けて確かめられる
- Optional は get() で取り出さずに、hasValue・isEmpty で判定する
- hasValueSatisfying：中身があることと、中身の値を、まとめて確かめる

unit-test/src/test/java/com/example/testing/basic/AssertionExamplesTest.java

アサーション ・ AssertJ

ソフトアサーション：失敗をまとめて報告する

```
Order order = new Order("A001", 2, 1980);

SoftAssertions.assertSoftly(softly -> {
    softly.assertThat(order.productCode()).isEqualTo("A001");
    softly.assertThat(order.quantity()).isEqualTo(2);
    softly.assertThat(order.totalPrice()).isEqualTo(1980);
});
```

ポイント

- 通常の assertThat は、最初に失敗したところでテストが止まる
- assertSoftly の中では、すべての判定を最後まで実行してから、失敗をまとめて報告する
- 1つのオブジェクトの複数の項目を確かめるときに便利

レコード同士なら、Assertions.assertThat(order).isEqualTo(new Order("A001", 2, 1980)) の1行でも確かめられます。

アサーション ・ AssertJ

よく使う判定の早見表

対象	判定
すべての値	isEqualTo / isNotEqualTo / isNull / isNotNull / isInstanceOf
数値	isPositive / isZero / isGreaterThan / isLessThanOrEqualTo / isBetween
文字列	isEmpty / isBlank / contains / startsWith / endsWith / matches
真偽値	isTrue / isFalse
リスト	hasSize / isEmpty / contains / containsExactly / doesNotContain / extracting
Optional	isPresent / isEmpty / hasValue / hasValueSatisfying
例外	Assertions.assertThatThrownBy(…).isInstanceOf / hasMessage / hasMessageContaining

Assertions.assertThat(…).isTrue() で何でも書けてしまいますが、失敗したときに「false だった」としかわかりません。目的に合った判定を選ぶと、失敗の理由がメッセージに表れます。

テスト対象：ほかの部品を使う OrderService

```
public class OrderService {
    // 在庫を調べる・減らす（データベース）、注文を通知する（メール）
    private final StockRepository stockRepository;
    private final PriceCalculator priceCalculator;
    private final OrderNotifier orderNotifier;

    public Order placeOrder(String productCode, int unitPrice, int quantity, MemberRank rank) {
        int stock = stockRepository.findStock(productCode);
        if (stock < quantity) {
            throw new OutOfStockException(productCode, stock, quantity);
        }
        stockRepository.decrease(productCode, quantity);

        int totalPrice = priceCalculator.calculate(unitPrice, quantity, rank);
        Order order = new Order(productCode, quantity, totalPrice);
        orderNotifier.notify(order);
        return order;
    }
}
```

StockRepository と OrderNotifier はインタフェースです。本物はデータベースやメールにつながるので、テストでは使いにくい部品です。

モック ・ Mockito

ドライバとスタブ：テスト対象を呼ぶもの、呼ばれるもの

テスト対象を単独で動かすには、テスト対象を呼び出すものと、テスト対象が呼び出す依存先の代わりが必要です。

役割	意味	単体テストでは
ドライバ	テスト対象を呼び出して動かす	テストコード（テストメソッド）
スタブ	テスト対象が呼び出す依存先の代わりをする	Mockito.mock(...) で作るモック

ポイント

・Mockito のモックは、決めた値を返す（when）ことも、呼び出しを記録して確かめる（verify）こともできる

・値を返すだけのものを「スタブ」、呼び出しを確かめるものを「モック」と呼び分けることもあるが、現場ではまとめて「モック」と呼ぶことが多い

モック ・ Mockito

モックを作り、テスト対象に渡す

```
@Test
@DisplayName("在庫が足りれば、在庫を減らして注文を通知する")
void placeOrder() {
    // Arrange：依存先をモックにして、テスト対象を組み立てる
    StockRepository stockRepository = Mockito.mock(StockRepository.class);
    OrderNotifier orderNotifier = Mockito.mock(OrderNotifier.class);
    OrderService orderService =
            new OrderService(stockRepository, new PriceCalculator(), orderNotifier);

    // Act・Assert は次のスライド
}
```

ポイント

- Mockito.mock(型.class) でモックを作る。mock・when・verify は、どれも Mockito クラスのメソッド
- 計算を受け持つ PriceCalculator は本物を、依存先の StockRepository・OrderNotifier はモックを渡す
- どのモックをどう渡したかが、そのテストを読むだけでわかる
- モックのメソッドは、何も決めていなければ 0・false・null・空のリストなどを返す

モック ・ Mockito

振る舞いを決め（when）、呼び出しを確かめる（verify）

```
@Test
@DisplayName("在庫が足りれば、在庫を減らして注文を通知する")
void placeOrder() {
    // Arrange
    StockRepository stockRepository = Mockito.mock(StockRepository.class);
    OrderNotifier orderNotifier = Mockito.mock(OrderNotifier.class);
    OrderService orderService =
            new OrderService(stockRepository, new PriceCalculator(), orderNotifier);
    Mockito.when(stockRepository.findStock("A001")).thenReturn(10);

    // Act
    Order order = orderService.placeOrder("A001", 1000, 2, MemberRank.GOLD);

    // Assert
    Assertions.assertThat(order).isEqualTo(new Order("A001", 2, 1980));
    Mockito.verify(stockRepository).decrease("A001", 2);
    Mockito.verify(orderNotifier).notify(order);
}
```

ポイント

- Mockito.when(モック.メソッド(引数)).thenReturn(値)：その引数で呼ばれたら、その値を返す
- Mockito.verify(モック).メソッド(引数)：その引数で1回呼ばれたことを確かめる

unit-test/src/test/java/com/example/testing/basic/OrderServiceTest.java

モック ・ Mockito

呼び出されないことを確かめる

```
@Test
@DisplayName("在庫が足りなければ例外になり、在庫も減らさず通知もしない")
void outOfStock() {
    // Arrange（モックの作成と、テスト対象の組み立ては省略）
    Mockito.when(stockRepository.findStock("A001")).thenReturn(1);

    Assertions.assertThatThrownBy(() -> orderService.placeOrder("A001", 1000, 2, MemberRank.REGULAR))
            .isInstanceOf(OutOfStockException.class);

    Mockito.verify(stockRepository, Mockito.never())
            .decrease(ArgumentMatchers.anyString(), ArgumentMatchers.anyInt());
    Mockito.verify(orderNotifier, Mockito.never()).notify(ArgumentMatchers.any());
}
```

書き方	意味
Mockito.verify(m, Mockito.never())	1回も呼ばれていない
Mockito.verify(m, Mockito.times(2))	ちょうど2回呼ばれた
ArgumentMatchers.any()	引数は何でもよい（型を決める anyString・anyInt もある）
thenThrow(例外)	呼ばれたら例外を投げる（when と組み合わせる）
Mockito.doThrow(例外).when(m).メソッド(…)	戻り値が void のメソッドで、例外を投げる（when(…) で囲めないため）

unit-test/src/test/java/com/example/testing/basic/OrderServiceTest.java

モック ・ Mockito

渡された引数の中身を確かめる：ArgumentCaptor

```
@Test
@DisplayName("通知した注文の中身を取り出して確かめる（ArgumentCaptor）")
void notifiedOrder() {
    // Arrange（モックの作成と、テスト対象の組み立ては省略）
    Mockito.when(stockRepository.findStock("A001")).thenReturn(10);

    orderService.placeOrder("A001", 1000, 3, MemberRank.SILVER);

    ArgumentCaptor<Order> captor = ArgumentCaptor.forClass(Order.class);
    Mockito.verify(orderNotifier).notify(captor.capture());
    Order notified = captor.getValue();
    Assertions.assertThat(notified.quantity()).isEqualTo(3);
    Assertions.assertThat(notified.totalPrice()).isEqualTo(3135);
}
```

ポイント

- ArgumentCaptor.forClass(型.class) で、引数を捕まえる入れ物を作る
- verify の引数に capture() を書くと、実際に渡された値を捕まえられる
- テスト対象の中で作られたオブジェクトを、あとから取り出して確かめたいときに使う

モック ・ Mockito

モックを使うときの注意

決めていない呼び出しは、既定値を返す

when を書き忘れると、findStock は 0 を返します。テストは在庫不足の例外で失敗し、原因がわかりにくくなります。テストに必要な振る舞いは、すべて when で書いておきます。

モックにするのは依存先

テスト対象が依存しているインスタンスをモックにします。OrderService のテストなら StockRepository、Controller のテストなら ApplicationService です。PriceCalculator のような計算や、Order のような値は本物を使います。

モックだらけのテストに注意

何でもモックにすると、「モックが返した値をそのまま確かめているだけ」のテストになります。確かめたいのはテスト対象の振る舞いです。

モック ・ Mockito

when で決めた呼び出しが使われたかを確かめる

when で振る舞いを決めても、テスト対象がそのメソッドを呼ばなければ、決めた振る舞いは使われないまま終わります。呼ばれたことも確かめたいときは、verify で明示的に書きます。

```
@Test
@DisplayName("注文を受け付ける前に、在庫を確認する")
void checksStock() {
    // Arrange（モックの作成と、テスト対象の組み立ては省略）
    Mockito.when(stockRepository.findStock("A001")).thenReturn(10);

    orderService.placeOrder("A001", 1000, 2, MemberRank.GOLD);

    // when で決めた呼び出しが、実際に使われたことを確かめる
    Mockito.verify(stockRepository).findStock("A001");
}
```

ポイント

- 戻り値を使う呼び出しは、多くの場合、その結果（注文の金額や例外など）で確かめられる
- 「在庫を確認せずに注文を受け付けてはいけない」のように、呼ばれること自体が仕様なら verify で確かめる

unit-test/src/test/java/com/example/testing/basic/OrderServiceTest.java

演習4 ・ モック

在庫ちょうどの注文をテストする

unit-test/src/test/java/com/example/testing/basic に OrderServiceExerciseTest を新しく作り、次のテストを書いてください。

・商品 "B002" の在庫が 5 個のとき、単価 200 円・数量 5 個・REGULAR で注文できる

・戻り値の Order が new Order("B002", 5, 1100) と等しい

・stockRepository の decrease が ("B002", 5) で呼ばれ、orderNotifier の notify がその Order で呼ばれる

ヒント：モックの作り方とテスト対象の組み立て方は OrderServiceTest と同じ（テストメソッドの Arrange で行う）

```
./mvnw test -pl unit-test -Dtest=OrderServiceExerciseTest
```

期待する結果

```
[INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0
```

第2部のまとめ

JUnit による基本的なテスト

JUnit：テストを組み立てて実行する

@Test・@DisplayName・@ParameterizedTest。テスト対象は各テストの Arrange で組み立て、テスト同士は独立させる。

パラメタライズドテスト：値だけを並べる

@ParameterizedTest と @CsvSource(delimiter = '|', textBlock = """ … """) で、入力と期待値を表のように並べる。

AssertJ：検査される値から書き始める

Assertions.assertThat(検査される値) に、目的に合った判定をつなげる。失敗の理由がメッセージに表れる。

Mockito：外側の部品を置き換える

when で振る舞いを決め、verify で呼び出しを確かめる。計算や値は本物を使う。

03

Spring プロジェクトのテスト

必要な部品だけを起動して、本物に近い環境で確かめる

Spring プロジェクトのテスト

@SpringBootTest と スライステスト

Spring のテストでは、アプリケーションの部品（Bean）を Spring に用意してもらいます。どこまでの部品を用意するかで、使うアノテーションが変わります。

アノテーション	用意する部品	確かめるもの
@SpringBootTest	アプリケーションのすべての部品	部品をつなげた全体の動き
@DataJdbcTest	Spring Data JDBC のリポジトリと JDBC の部品	リポジトリのクエリとテーブルの対応づけ
@DataJpaTest	JPA の部品と Spring Data JPA のリポジトリ	エンティティの対応づけとクエリ
@WebMvcTest	コントローラーと Spring MVC の部品	URL・リクエスト・レスポンス

ポイント

- @DataJdbcTest などは、アプリケーションの一部だけを切り出すので「スライステスト」と呼ぶ
- 用意する部品が少ないほど起動が速く、テストの失敗の原因も絞り込みやすい

Testcontainers を使うための依存関係

```
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-testcontainers</artifactId>
    <scope>test</scope>
</dependency>
<dependency>
    <groupId>org.testcontainers</groupId>
    <artifactId>testcontainers-junit-jupiter</artifactId>
    <scope>test</scope>
</dependency>
<dependency>
    <groupId>org.testcontainers</groupId>
    <artifactId>testcontainers-postgresql</artifactId>
    <scope>test</scope>
</dependency>
```

ポイント

- spring-boot-testcontainers：@ServiceConnection で、Spring Boot と Testcontainers をつなぐ
- testcontainers-junit-jupiter：@Testcontainers・@Container で、JUnit からコンテナを起動・停止する
- testcontainers-postgresql・testcontainers-kafka：PostgreSQL・Kafka 用のコンテナのクラス
- バージョンは Spring Boot が管理しているので、書かなくてよい

spring-test/pom.xml

Spring プロジェクトのテスト

Testcontainers のアノテーション

```
@Testcontainers
class ProductRepositoryTest {

    @Container
    @ServiceConnection
    static PostgreSQLContainer postgres = new PostgreSQLContainer("postgres:17-alpine");
}
```

アノテーション	役割
@Testcontainers	このクラスの @Container を探し、起動・停止を JUnit に任せる
@Container	管理してほしいコンテナの目印。static フィールドなら、クラスの最初に起動し最後に停止する
@ServiceConnection	コンテナの接続先を、Spring Boot の接続設定（URL・ユーザー名など）として使わせる

@ServiceConnection のおかげで、application.properties の spring.datasource.url などを、テスト用に書き換えなくて済みます。

Spring プロジェクトのテスト

テーブルは schema.sql で作り、データはテストごとに登録する

```
create table if not exists products (
    id    bigint generated by default as identity primary key,
    name  varchar(100) not null,
    price integer      not null,
    stock integer      not null
);
```

```
# schema.sql でテーブルを作る（JPA にはテーブルを作らせない）
spring.sql.init.mode=always
spring.jpa.hibernate.ddl-auto=none
```

ポイント

- Spring Boot は起動時に src/main/resources/schema.sql を実行する。コンテナの空のデータベースにも、テーブルが作られる
- テストデータは、テストメソッドごとに登録する（JDBC・JPA の単元で、それぞれの登録のしかたを扱う）
- テスト対象のメソッド（save など）でテストデータを登録しない。そのメソッドが壊れていると、ほかのテストまで失敗してしまう

spring-test/src/main/resources/schema.sql
spring-test/src/main/resources/application.properties

JDBC のテスト

テスト対象：Spring Data JDBC の ProductRepository

```
@Table("products")
public record Product(@Id Long id, String name, int price, int stock) {

    public Product addStock(int quantity) {
        return new Product(id, name, price, stock + quantity);
    }
}
```

```
public interface ProductRepository extends ListCrudRepository<Product, Long> {

    /** 単価が maxPrice 以下の商品を、単価の安い順に返す（メソッド名から SQL が作られる） */
    List<Product> findByPriceLessThanEqualOrderByPriceAsc(int maxPrice);
}
```

ポイント

- リポジトリは、商品をコレクションのように出し入れする窓口。save・findById などは ListCrudRepository が用意する
- insert・update といった SQL の都合は、メソッド名に出さない。保存はすべて save
- テーブルとの対応づけやクエリが正しいかは、実際のデータベースで動かして確かめる

spring-test/src/main/java/com/example/testing/product/Product.java
spring-test/src/main/java/com/example/testing/product/ProductRepository.java

JDBC のテスト

@DataJdbcTest で、Spring Data JDBC の部品とコンテナを用意する

```
@DataJdbcTest
@Testcontainers
class ProductRepositoryTest {

    @Container
    @ServiceConnection
    static PostgreSQLContainer postgres = new PostgreSQLContainer("postgres:17-alpine");

    @Autowired
    private ProductRepository productRepository;

    @Autowired
    private JdbcClient jdbcClient; // 検証用：リポジトリを通さずにテーブルを読む
}
```

ポイント

・@DataJdbcTest：Spring Data JDBC のリポジトリと JDBC の部品だけを用意する。テストごとにトランザクションを始め、最後にロールバックする

・@ServiceConnection のコンテナは、テスト用のデータベースとして扱われ、組み込みデータベースに置き換えられない

・以前は @AutoConfigureTestDatabase(replace = NONE) を付ける必要があったが、今の Spring Boot では要らない

spring-test/src/test/java/com/example/testing/product/ProductRepositoryTest.java

JDBC のテスト

@Sql でテストデータを用意して、検索を確かめる

```
@Test
@Sql(statements = """
        insert into products (id, name, price, stock) values (101, 'ボールペン', 120, 10);
        """)
@DisplayName("IDで商品を取得できる")
void findById() {
    Optional<Product> found = productRepository.findById(101L);

    Assertions.assertThat(found).hasValue(new Product(101L, "ボールペン", 120, 10));
}
```

ポイント

- @Sql は、テストメソッドの前に SQL を実行する。statements に SQL をそのまま書ける
- テストデータを、テスト対象のリポジトリ（save）で登録しない。save が壊れていると、findById のテストまで失敗してしまう
- ID を決めて登録するので、期待値をそのまま書ける

spring-test/src/test/java/com/example/testing/product/ProductRepositoryTest.java

JDBC のテスト

検索条件と並び順を確かめる

```
@Test
@Sql(statements = """
        insert into products (id, name, price, stock) values (101, 'ノート', 200, 30);
        insert into products (id, name, price, stock) values (102, 'ボールペン', 120, 10);
        insert into products (id, name, price, stock) values (103, '万年筆', 5000, 2);
        """)
@DisplayName("指定した単価以下の商品を、安い順に返す")
void findByPriceLessThanEqual() {
    List<Product> products = productRepository.findByPriceLessThanEqualOrderByPriceAsc(200);

    Assertions.assertThat(products)
            .extracting(Product::name)
            .containsExactly("ボールペン", "ノート");
}
```

ポイント

- 200 円ちょうどのノートが含まれるか（境界）と、並び順を containsExactly でまとめて確かめる
- 5,000 円の万年筆が含まれないことも、containsExactly で確かめられる

spring-test/src/test/java/com/example/testing/product/ProductRepositoryTest.java

JDBC のテスト

save を確かめる：結果はリポジトリを通さずに読む

```
@Test
@DisplayName("新しい商品を保存すると、IDが採番されてテーブルに登録される")
void saveNew() {
    Product saved = productRepository.save(new Product(null, "ボールペン", 120, 10));

    Product row = jdbcClient.sql("select id, name, price, stock from products where id = :id")
            .param("id", saved.id())
            .query(Product.class)
            .single();
    Assertions.assertThat(row).isEqualTo(new Product(saved.id(), "ボールペン", 120, 10));
}
```

ポイント

- 確かめたいのは save なので、結果の確認に findById は使わない。JdbcClient でテーブルを直接読む
- 既存の商品の更新は、@Sql で登録した行を save で書き換え、JdbcClient で読んで確かめる（サンプルの saveExisting）

spring-test/src/test/java/com/example/testing/product/ProductRepositoryTest.java

JDBC のテスト

テストが終わると、登録したデータは消える

@DataJdbcTest のテスト1つの流れ

```
テスト開始          →  トランザクション開始
@Sql                →  商品を登録
テストメソッド      →  findById で確かめる
テスト終了          →  ロールバック（登録したデータは残らない）
```

ポイント

- @DataJdbcTest・@DataJpaTest は、テストメソッドごとにトランザクションを始め、最後にロールバックする
- @Sql の SQL も同じトランザクションの中で実行されるので、テストが終われば消える
- あるテストで登録したデータが、別のテストの検索結果に混ざらない

@SpringBootTest は、何も指定しないとロールバックしません。KafkaListener のテストでは、この違いを意識します。

演習5 ・ JDBC のテスト

在庫の少ない商品を探す検索をテストする

テスト対象：ProductRepository に追加した、次のメソッドです。

```
/** 在庫数が threshold 未満の商品を、在庫の少ない順に返す（演習5のテスト対象） */
List<Product> findByStockLessThanOrderByStockAsc(int threshold);
```

ProductRepositoryExerciseTest に、次の2つのテストメソッドを書いてください。テストデータは @Sql で登録します。

- 在庫数が 0・9・10・20 の4つの商品を登録すると、findByStockLessThanOrderByStockAsc(10) が在庫 0・9 の商品をこの順に返す
- 在庫数が 10・20 の2つの商品を登録すると、findByStockLessThanOrderByStockAsc(10) が空のリストを返す

ファイル：spring-test/src/test/java/com/example/testing/product/ProductRepositoryExerciseTest.java（コンテナとリポジトリの用意は書いてあります）

```
./mvnw test -pl spring-test -Dtest=ProductRepositoryExerciseTest
```

期待する結果

```
[INFO] Tests run: 2, Failures: 0, Errors: 0, Skipped: 0
```

spring-test/src/main/java/com/example/testing/product/ProductRepository.java

JPA のテスト

テスト対象：Member エンティティと MemberRepository

```
@Entity
@Table(name = "members")
public class Member {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    private String name;
    private String email;
    @Enumerated(EnumType.STRING)
    @Column(name = "member_rank")
    private MemberRank rank;
    // コンストラクタ・getter・changeRank は省略
}
```

```
public interface MemberRepository extends JpaRepository<Member, Long> {
    Optional<Member> findByEmail(String email);
    List<Member> findByRankOrderByNameAsc(MemberRank rank);

    @Query("select m from Member m where m.name like concat('%', :keyword, '%') order by m.id")
    List<Member> searchByName(@Param("keyword") String keyword);
}
```

spring-test/src/main/java/com/example/testing/member/Member.java
spring-test/src/main/java/com/example/testing/member/MemberRepository.java

JPA のテスト

@DataJpaTest で、JPA の部品とコンテナを用意する

```
@DataJpaTest
@Testcontainers
class MemberRepositoryTest {

    @Container
    @ServiceConnection
    static PostgreSQLContainer postgres = new PostgreSQLContainer("postgres:17-alpine");

    @Autowired
    private MemberRepository memberRepository;

    @Autowired
    private TestEntityManager entityManager; // テストデータの登録用
}
```

ポイント

- @DataJpaTest は、Spring Data のリポジトリ（MemberRepository）と TestEntityManager を用意する
- テストデータは、リポジトリの save ではなく TestEntityManager で登録する
- テストごとにロールバックするのは @DataJdbcTest と同じ

spring-test/src/test/java/com/example/testing/member/MemberRepositoryTest.java

JPA のテスト

TestEntityManager で登録し、flush と clear をする

JPA は、登録したエンティティを手元（永続化コンテキスト）に覚えています。そのまま検索すると、データベースに問い合わせず、覚えているものを返すことがあります。

```
// Arrange：テストデータを登録し、SQL を送ってから、JPA の記憶を消す
entityManager.persist(new Member("佐藤", "sato@example.com", MemberRank.GOLD));
entityManager.flush();
entityManager.clear();

// Act
Optional<Member> found = memberRepository.findByEmail("sato@example.com");

// Assert
Assertions.assertThat(found).hasValueSatisfying(m -> {
    Assertions.assertThat(m.getName()).isEqualTo("佐藤");
    Assertions.assertThat(m.getRank()).isEqualTo(MemberRank.GOLD);
});
```

メソッド	役割
persist	エンティティを登録する（この時点では SQL が送られないことがある）
flush	たまっている SQL をデータベースへ送る
clear	覚えているエンティティを忘れる。このあとの検索は、必ずデータベースから読む

spring-test/src/test/java/com/example/testing/member/MemberRepositoryTest.java

JPA のテスト

派生クエリを確かめる

```
@Test
@DisplayName("ランクで絞り込み、名前の順に並べる")
void findByRankOrderByNameAsc() {
    entityManager.persist(new Member("鈴木", "suzuki@example.com", MemberRank.SILVER));
    entityManager.persist(new Member("伊藤", "ito@example.com", MemberRank.SILVER));
    entityManager.persist(new Member("高橋", "takahashi@example.com", MemberRank.GOLD));
    entityManager.flush();
    entityManager.clear();

    List<Member> members = memberRepository.findByRankOrderByNameAsc(MemberRank.SILVER);

    Assertions.assertThat(members)
            .extracting(Member::getEmail)
            .containsExactly("ito@example.com", "suzuki@example.com");
}
```

ポイント

- GOLD の高橋さんが含まれないこと（絞り込み）と、並び順を確かめている
- 日本語の並び順はデータベースの照合順序（collation）に左右される。並び順を確かめるときは、環境によって結果が変わらないか確かめておく

spring-test/src/test/java/com/example/testing/member/MemberRepositoryTest.java

JPA のテスト

@Query と一意制約を確かめる

```
@Test
@DisplayName("名前に keyword を含む会員を返す（@Query）")
void searchByName() {
    entityManager.persist(new Member("田中一郎", "ichiro@example.com", MemberRank.REGULAR));
    entityManager.persist(new Member("田中花子", "hanako@example.com", MemberRank.GOLD));
    entityManager.persist(new Member("渡辺", "watanabe@example.com", MemberRank.REGULAR));
    entityManager.flush();
    entityManager.clear();

    List<Member> members = memberRepository.searchByName("田中");

    Assertions.assertThat(members).extracting(Member::getName).containsExactly("田中一郎", "田中花子");
}

@Test
@DisplayName("同じメールアドレスは登録できない（一意制約）")
void duplicateEmail() {
    entityManager.persistAndFlush(new Member("佐藤", "sato@example.com", MemberRank.REGULAR));

    Assertions.assertThatThrownBy(() -> memberRepository.saveAndFlush(
            new Member("佐藤次郎", "sato@example.com", MemberRank.GOLD)))
            .isInstanceOf(DataIntegrityViolationException.class);
}
```

ポイント

・persistAndFlush は persist と flush を続けて行う。saveAndFlush も save のあとすぐに SQL を送るので、一意制約の違反がその場で例外になる

spring-test/src/test/java/com/example/testing/member/MemberRepositoryTest.java

演習6 ・ JPA のテスト

メールアドレスが登録済みかを確かめる

テスト対象：MemberRepository に追加した、次のメソッドです。

```
/** メールアドレスが登録済みかを返す（派生クエリ・演習6のテスト対象） */
boolean existsByEmail(String email);
```

MemberRepositoryExerciseTest に、次の2つのテストメソッドを書いてください。会員は TestEntityManager で登録し、flush と clear をしてから確かめます。

・会員（佐藤・sato@example.com・GOLD）を登録すると、existsByEmail("sato@example.com") が true を返す

・同じ会員を登録しても、existsByEmail("suzuki@example.com") は false を返す

ファイル：spring-test/src/test/java/com/example/testing/member/MemberRepositoryExerciseTest.java（コンテナ・リポジトリ・TestEntityManager の用意は書いてあります）

```
./mvnw test -pl spring-test -Dtest=MemberRepositoryExerciseTest
```

期待する結果

```
[INFO] Tests run: 2, Failures: 0, Errors: 0, Skipped: 0
```

RestController のテスト

テスト対象：商品の REST API

```
@RestController
@RequestMapping("/products")
public class ProductController {
    private final ProductService productService;

    @GetMapping("/{id}")
    public ResponseEntity<Product> lookup(@PathVariable("id") long id) {
        return productService.lookup(id)
                .map(ResponseEntity::ok)
                .orElseGet(() -> ResponseEntity.notFound().build());
    }

    @PostMapping
    public ResponseEntity<Product> register(@RequestBody ProductRegisterRequest request,
                                            UriComponentsBuilder uriBuilder) { ... } // 201 と Location

    @ExceptionHandler(IllegalArgumentException.class)
    public ResponseEntity<Void> handleIllegalArgument() {
        return ResponseEntity.badRequest().build(); // 400
    }
}
```

確かめたいのは、URL と HTTP メソッド・ステータスコード・JSON の形です。商品の検索や登録の処理そのものは、ここでは確かめません。

spring-test/src/main/java/com/example/testing/product/ProductController.java

RestController のテスト

@MockitoBean：Spring の部品をモックに差し替える

Mockito.mock(...)

特徴

- Mockito がモックを作るだけ
- Spring は関係しない。テスト対象は自分で new して、モックを渡す
- 第2部の単体テストで使った

@MockitoBean（Spring）

特徴

- モックを作り、Spring の部品として登録する
- 同じ型の本物の部品の代わりに、テスト対象へ注入される
- @WebMvcTest などのスライステストで使う

```
@MockitoBean
private ProductService productService; // ProductController にはこのモックが注入される
```

以前は Spring Boot の @MockBean を使っていましたが、Spring Boot 4.0 で削除されました。古い記事の @MockBean は @MockitoBean に読み替えます。

RestController のテスト

@WebMvcTest と MockMvcTester

```
@WebMvcTest(ProductController.class)
class ProductControllerTest {
    @Autowired
    private MockMvcTester mvc;
    @MockitoBean
    private ProductService productService;

    @Test
    @DisplayName("GET /products/{id}：見つかれば 200 と商品の JSON")
    void lookup() {
        Mockito.when(productService.lookup(1L))
                .thenReturn(Optional.of(new Product(1L, "ボールペン", 120, 10)));

        MvcTestResult result = mvc.get().uri("/products/{id}", 1).exchange();

        Assertions.assertThat(result)
                .hasStatusOk()
                .bodyJson()
                .isStrictlyEqualTo("""
                        {"id": 1, "name": "ボールペン", "price": 120, "stock": 10}
                        """);
    }
}
```

ポイント

・MockMvcTester：AssertJ の書き方でリクエストと結果を扱える。exchange() で送り、結果を assertThat で確かめる

spring-test/src/test/java/com/example/testing/product/ProductControllerTest.java

RestController のテスト

404 と 201 を確かめる

```
Mockito.when(productService.lookup(99L)).thenReturn(Optional.empty());

MvcTestResult result = mvc.get().uri("/products/{id}", 99).exchange();

Assertions.assertThat(result).hasStatus(HttpStatus.NOT_FOUND);
```

```
Mockito.when(productService.register("ノート", 200, 30)).thenReturn(new Product(2L, "ノート", 200, 30));

MvcTestResult result = mvc.post().uri("/products")
        .contentType(MediaType.APPLICATION_JSON)
        .content("""
                {"name": "ノート", "price": 200, "stock": 30}
                """)
        .exchange();

Assertions.assertThat(result)
        .hasStatus(HttpStatus.CREATED)
        .hasHeader("Location", "http://localhost/products/2");
```

ポイント

・入力値が不正なときの 400 も、同じ形で hasStatus(HttpStatus.BAD_REQUEST) を確かめる（サンプルの registerBadRequest）

spring-test/src/test/java/com/example/testing/product/ProductControllerTest.java

RestController のテスト

MockMvcTester の書き方

書き方	意味
mvc.get().uri("/products/{id}", 1)	GET リクエストを組み立てる（post・put・delete も同じ形）
.contentType(…).content(…)	リクエストボディを付ける
.exchange()	リクエストを送り、結果（MvcTestResult）を受け取る
Assertions.assertThat(result).hasStatusOk()	ステータスコードを確かめる（hasStatus(HttpStatus.CREATED) など）
.hasHeader(名前, 値)	レスポンスヘッダーを確かめる
.bodyJson().isStrictlyEqualTo(…)	レスポンスの JSON 全体を確かめる
.bodyJson().extractingPath("$.name")	JSON の一部を取り出して確かめる

ポイント

- 本物のサーバーを起動せず、Spring MVC にリクエストを直接渡すので速い
- 既存のコードでは、MockMvc の perform(…).andExpect(…) で書かれたテストもよく見かける。読めるようにしておく

演習7 ・ RestController のテスト

入荷 API をテストする

テスト対象：ProductController に追加した、入荷を受け付ける API です。ProductNotFoundException は、@ExceptionHandler で 404 に変換されます。

```
@PostMapping("/{id}/arrivals")
public ResponseEntity<Void> arrive(@PathVariable("id") long id,
                                   @RequestBody StockArrivalRequest request) {
    productService.addStock(id, request.quantity());
    return ResponseEntity.noContent().build();
}
```

ProductControllerExerciseTest に、次の2つのテストメソッドを書いてください。ProductService は @MockitoBean のモックです。

- POST /products/1/arrivals に {"quantity": 5} を送ると 204 が返り、productService.addStock(1L, 5) が呼ばれる
- addStock(99L, 5) が ProductNotFoundException を投げるとき、POST /products/99/arrivals は 404 を返す
- ヒント：戻り値が void のメソッドは Mockito.doThrow(…).when(productService).addStock(…) で例外を投げさせる

ファイル：spring-test/src/test/java/com/example/testing/product/ProductControllerExerciseTest.java

```
./mvnw test -pl spring-test -Dtest=ProductControllerExerciseTest
```

KafkaListener のテスト

テスト対象：入荷の知らせを受け取るリスナー

```
@Component
public class StockArrivalListener {
    private final ProductService productService;

    @KafkaListener(topics = "${spring.kafka.template.default-topic}")
    public void onMessage(ConsumerRecord<String, String> record) {
        long productId = Long.parseLong(record.key()); // キー：商品ID
        int quantity = Integer.parseInt(record.value()); // 値：入荷数
        productService.addStock(productId, quantity);
    }
}
```

```
# application.properties
spring.kafka.template.default-topic=stock-arrivals
```

ポイント

- トピック名はコードに書かず、${…} で設定値（spring.kafka.template.default-topic）を参照する
- 同じ設定値が KafkaTemplate の既定のトピックにもなるので、テストでは sendDefault でこのトピックへ送れる
- メッセージを受け取ってデータベースを更新するまでを、1つのテストで確かめる

spring-test/src/main/java/com/example/testing/product/StockArrivalListener.java
spring-test/src/main/resources/application.properties

KafkaListener のテスト

EmbeddedKafka を使わず、本物の Kafka で確かめる

EmbeddedKafka

特徴

・spring-kafka-test が提供する。テストと同じ JVM の中で Kafka を動かす

・Docker が要らない

・Kafka のバージョンや設定が、本番のブローカーと同じとは限らない

Testcontainers の KafkaContainer

特徴

・apache/kafka-native の Docker イメージを起動する。Kafka をネイティブイメージにしたもので、起動が速い

・Docker が必要

・本番と同じ Kafka のバージョンを指定でき、PostgreSQL と同じ書き方でそろえられる

データベースも Kafka も「本番と同じソフトウェアをコンテナで起動する」に統一すると、テストの書き方も1つにそろいます。

KafkaListener のテスト

@SpringBootTest と、2つのコンテナ

```
@SpringBootTest
@Testcontainers
class StockArrivalListenerTest {

    @Container
    @ServiceConnection
    static PostgreSQLContainer postgres = new PostgreSQLContainer("postgres:17-alpine");
    @Container
    @ServiceConnection
    static KafkaContainer kafka = new KafkaContainer("apache/kafka-native:4.0.0");

    @Autowired
    private KafkaTemplate<String, String> kafkaTemplate; // 既定のトピックへ送る
    @MockitoSpyBean
    private StockArrivalListener stockArrivalListener; // 処理の終わりを知るためのスパイ
    @Autowired
    private JdbcClient jdbcClient; // 検証用：テーブルを直接読む
}
```

ポイント

・リスナー・サービス・リポジトリをつなげて動かすので、@SpringBootTest ですべての部品を用意する

・@MockitoSpyBean：本物の Bean を、本物を包むスパイに差し替える。振る舞いは本物のまま、呼び出しに処理を足せる

spring-test/src/test/java/com/example/testing/product/StockArrivalListenerTest.java

KafkaListener のテスト

CountDownLatch：処理が終わった時点で待ちを解除する

待ち方	待つ時間	問題
Thread.sleep	決めた時間を必ず待つ	速く終わっても待つ。遅い環境では足りず、たまに失敗する
Awaitility の untilAsserted	条件がそろうまで、間をあけて確かめ直す	そろうまで、アサーション（テーブルの読み取り）が何度も走る
CountDownLatch	countDown されるまで	なし。処理が終わった時点で、1回だけ確かめる

```
CountDownLatch processed = new CountDownLatch(1); // 数を決める
processed.countDown(); // 1 減らす（0 になると待ちが解除される）
processed.await(10, TimeUnit.SECONDS); // 0 になるまで待つ。時間切れなら false
```

非同期の処理を待つときは、時間で待たずに、処理の終わりを知らせる仕組み（CountDownLatch など）を用意して待ちます。

spring-test/src/test/java/com/example/testing/product/StockArrivalListenerTest.java

KafkaListener のテスト

リスナーの処理が終わるのを待って、確かめる

```
@Test
@Sql(statements = """
        insert into products (id, name, price, stock) values (201, 'ボールペン', 120, 10);
        """)
@DisplayName("入荷メッセージを受け取ると、在庫数が増える")
void stockArrival() throws InterruptedException {
    // Arrange：リスナーの本物の処理が終わったら countDown する
    CountDownLatch processed = new CountDownLatch(1);
    Mockito.doAnswer(invocation -> {
        invocation.callRealMethod();
        processed.countDown();
        return null;
    }).when(stockArrivalListener).onMessage(ArgumentMatchers.any());

    // Act
    kafkaTemplate.sendDefault("201", "5");

    // Assert：処理が終わるまで待ってから、テーブルを確かめる
    Assertions.assertThat(processed.await(10, TimeUnit.SECONDS)).isTrue();
    Integer stock = jdbcClient.sql("select stock from products where id = 201")
            .query(Integer.class).single();
    Assertions.assertThat(stock).isEqualTo(15);
}
```

ポイント

・Mockito.doAnswer(処理).when(スパイ).メソッド(…)：呼ばれたときの処理を決める。callRealMethod() で本物を実行してから countDown する

spring-test/src/test/java/com/example/testing/product/StockArrivalListenerTest.java

第3部のまとめ

確かめたい層に合わせて、アノテーションを選ぶ

確かめたいもの	アノテーション	テストデータ	コンテナ
Spring Data JDBC のクエリ	@DataJdbcTest	@Sql	PostgreSQL
JPA の対応づけとクエリ	@DataJpaTest	TestEntityManager	PostgreSQL
REST API の入口と出口	@WebMvcTest	サービスは @MockitoBean	なし
Kafka からデータベースまで	@SpringBootTest	@Sql	PostgreSQL・Kafka

ポイント

- スライステストで、必要な部品だけを起動する
- データベースやメッセージングは、Testcontainers で本番と同じソフトウェアを使う
- テストデータは @Sql や TestEntityManager で登録し、テスト対象のメソッドに頼らない
- REST API は MockMvcTester で、AssertJ の書き方で確かめる
- 非同期の処理は、CountDownLatch で処理の終わりを待ってから確かめる

まとめ

テストコードは、変更を支える仕組み

第1部：考え方と道具

テストは動きを確かめ続けるためのコード。JUnit・AssertJ・Mockito の組み合わせが、Spring Boot の標準でもあり、現場の定番。

第2部：テストの書き方

準備・実行・検証の形で、1つの振る舞いを確かめる。境界の両側を選び、パラメタライズドテストで並べ、外側の部品だけをモックにする。

第3部：Spring のプロジェクト

スライステストで確かめたい層だけを起動し、Testcontainers で本物のデータベースや Kafka を使う。

コードを書いたら、テストも書く。テストが通る状態を保ちながら、少しずつ変更していきましょう。